字符串方法

var str1 = 'candy'; //字符串的字面量/直接量;

var str2 = String(); //函数式 底层API方法

var str3 = new String('abc123'); //构造函数 对象创建;

var str3 = '烂不烂问厨房';

console.log(str3.length) //length长度

console.log(str3.charAt(1)) //'不' charAt 指定索引的字符串;

console.log(str3 + ', 问我做甚?')

console.log(str3.concat(', 问我做甚? ')) //concat合并字符串;

console.log(str3.concat(', 问我做甚? ',"老夫又不知道!!!" ,"???")) //可以拼接多条信息

console.log(str3.indexOf('问')) //3 返回所匹配的下标;

console.log(str3.indexOf('沙漠')) //返回值:-1说明没有查找到 或者返回找到的

console.log(str6.indexOf('烂' ,1)) //indexOf的第二参数是从索引值指定的位置开始进行匹配查寻;

charCodeAt(index)返回指定索引的ASCII编码

```
//返回指定索引的ASCII编码
console.log(str2.charCodeAt(1)) // 50    //ASCII表;
```

fromCharCode:返回指定ASCII编码的字符串

```
//从字符编码创建一个字符串。一个或多个 ASCII编码的 值
console.log( String.fromCharCode(53) );
console.log(String.fromCharCode(53,50,48,49,51,49,52));  //要在内建函数里面去调用;
```

var str4 ="5201314"

console.log(str4.substring(0,5)) //"52013" ;

console.log(str4.substring(2,5)) //"013" ; 截取字符串,从开始的值切,到结束的值;[s下标,e下标)

console.log(str4.substr(2 , 5)) //'01314'截取字符串,从下标开始切,后面的切的长度;

console.log(str4.slice(2 , 5)) //"013"截取字符串,同substring的切法一样,从左往右切的;

console.log(str4.slice(0, -1)) //5201 index:开始位置的索引 count:要删除元素的个数 返回:返回的是包含被删除元素的数组对象

```
        //( index , count )  count为正数的时候，从左往右切；count为负数的时候，从右往左切；
```

字母大小写转换

```
var str5 = "JAVASCRIPT";
        console.log( str5.toLocaleLowerCase() )  //大写转小写
        console.log( str5.toLowerCase() )  //大写转小写
        var str6 = "javascript";
        console.log( str6.toLocaleUpperCase() )  //小写转大写;
        console.log( str6.toUpperCase() )  //小写转大写;

        console.log( str6.match('s') ) //匹配字符串的指定的值，返回匹配值的结果;
```

```
▼["s", index: 4, input: "javascript", groups: undefined]
   0: "s"
   groups: undefined
   index: 4
   input: "javascript"
   length: 1
 ▶ __proto__: Array(0)
```

```
        console.log( str6.match('e') ) //匹配不到值的情况，返回null;
```

替换

```
console.log( str6.replace( 'a' ,'sum! ! ! ') )
```

切割字符串的方法

切割字符串，返回数组

```
console.log( str6.split( '' ) )  //["j", "a", "v", "a", "s", "c", "r", "i", "p", "t"]
console.log( str6.split( 'a' ) ) //["j", "v", "script"]
console.log( str6.split( 'a' , 1) ) // ["j"]     第二个值控制返回的长度
```

数组的方法

创建数组的的三种方式：

```
var arr = [] ;  //
        var arr1 = Array( 1 , [666,false] , 'summy' )
        var arr2 = new Array( 1 , [666,false] , 'summy' )
        var arr3 = Array( 8 )  //创建一个叫【8】，生成的时候却是创建长度为8的空数组;
console.log(  arr3.length )  //读;
        arr3[3] = 123 ;  //写
```

数组的拼接方法：

// cancat 不改变原数组。concat合并数组之后，返回值才是新数组，并且可以合并两个及其以上的数组
console.log(arr4.concat('1314')) //["5", "2", "0", "1314"]

//.push.apply合并数组是把后一个数组的值依次push进前一个数组，使前一个数组发生改变，

//并且只能两个数组之间发生合并。
arr1.push.apply(arr1,arr2)
console.log(arr1) //[1, 2, 3, 4, 4, 5, 6, 7]

```
var arr6 = ["j", "a", "v", "a", "s", "c", "r", "i", "p", "t"];
        console.log( arr6.join() )    //数组转字符串;
                                      //j,a,v,a,s,c,r,i,p,t
        console.log( arr6.join('') )//javascript
        console.log( arr6.join('-') )//j-a-v-a-s-c-r-i-p-t
        console.log( arr6.join('*') )//j*a*v*a*s*c*r*i*p*t
```

找到对应的下标

```
console.log( arr6.indexOf('a') )//下标1
console.log( arr6.indexOf('a',2) )//下标3
```

删除数组的第一个值:

```
arr6.shift()//会自动补齐位置
```

在数组的头部添加值:

```
arr6.unshift('sum','ss')//
```

删除数组的最后一个值:

```
arr6.pop();
```

在数组的尾部添加值:

```
arr6.push('666',"11")
```

在数组的后面添加值:

```
arr6[arr6.length] = '333'
```

删除数组的某个对应下标的值，并且会留空empty:

```
delete arr6[0];
```

向数组的中间添加值:

```
var arr = [5,2,0,1,3,1,4];
console.log( arr.splice(1,5) )  //返回删除值[2, 0, 1, 3, 1]
```

解释: arr.splice(参数一, 参数二); 参数一: 从下标开始切 //参数二: 切的长度;参数三: 添加值

用于排序: sort()方法

如果字符串内有数字和字符串混排则只会排序数字，字符串位置不变。

```
<script type="text/javascript">

var arr = new Array(6)
arr[0] = "George"
arr[1] = "John"
arr[2] = "Thomas"
arr[3] = "James"
arr[4] = "Adrew"
arr[5] = "Martin"

document.write(arr + "<br />")
document.write(arr.sort())

</script>
```

输出:

```
George,John,Thomas,James,Adrew,Martin
Adrew,George,James,John,Martin,Thomas
```

在本例中，我们将创建一个数组，并按字母顺序进行排序:

```
<script type="text/javascript">

var arr = new Array(6)
arr[0] = "10"
arr[1] = "5"
arr[2] = "40"
arr[3] = "25"
arr[4] = "1000"
arr[5] = "1"

document.write(arr + "<br />")
document.write(arr.sort())

</script>
```

输出:

```
10,5,40,25,1000,1
1,10,1000,25,40,5
```

- 若 a 小于 b，在排序后的数组中 a 应该出现在 b 之前，则返回一个小于 0 的值。
- 若 a 等于 b，则返回 0。
- 若 a 大于 b，则返回一个大于 0 的值。

```
<script type="text/javascript">

function sortNumber(a,b)
{
return a - b
}

var arr = new Array(6)
arr[0] = "10"
arr[1] = "5"
arr[2] = "40"
arr[3] = "25"
arr[4] = "1000"
arr[5] = "1"

document.write(arr + "<br />")
document.write(arr.sort(sortNumber))

</script>
```

输出:

```
10,5,40,25,1000,1
1,5,10,25,40,1000
```

数组中slice（）的用法

```
var arr = [5,2,0,1,3,1,4];
var arr1 = arr.slice( 1 , 5 )//[2, 0, 1, 3]
```

slice(下标,从头开始数个数);(数组提取元素,左闭右开，不会修改原数组)

反转数组

```
arr.reverse()//[4, 1, 3, 1, 0, 2, 5]
```

判断元素是否为数组对象

是的话就返回true,否则反之;

```
 console.log( Array.isArray(arr) )//true
 console.log( Array.isArray({}) )//false
```

map()映射

```
//map映射;  每一项，索引 ,  数组本身
        var arr2 =  arr.map(function( value , index , self ){
            // console.log( value )
            // console.log( index )
            // console.log( self )

            return value*3;

        })
        console.log( arr2 )//[12, 3, 9, 3, 0, 6, 15]
```

forEach 遍历数组的方法:

```html
<button>1</button>
<button>2</button>
<button>3</button>
```

```
        //forEach 遍历数组的方法;
        arr4.forEach(function(val,idx,self){
            // console.log( val )
            val.onclick = function(){
                console.log( idx )
            }

        })

        //for(){ obj.i = i }
```

filter进行判断，返回出满足条件的数组对象；

```
var arr = [1,2,"4",false , function()
{console.log(1)},-1,999];

//filter进行判断，返回出满足条件的数组对象；
var isNum = arr.filter(function( val , idx , self ){

    return  typeof val === "number";

})
console.log( isNum )//[1, 2, -1, 999]
var isNum1 = arr.filter(function( val , idx , self ){

    return  typeof val !== "number";

})
console.log( isNum1 )//["4", false, ƒ]
```

find:进行判断,返回出满足条件的数组的第一项:

```
var isNotNum1 = arr.find(function( val , idx , self )
{

    return  typeof val !== "number";

})
console.log( isNotNum1 )
```